

zswap 內容重複率量測研究

容器化與虛擬化環境下的 Swap 內容去重特性分析

期末專案研究報告

量測環境：容器（Linux kernel 6.8, Ubuntu 24.04）／ 虛擬機（KVM, Ubuntu Cloud Image）

共用 eBPF kprobe 動態追蹤工具（zswap_store 進入點）／ 共用自製壓力程式 stress

本研究將程式碼放入：[https://github.com/92Jay0810/zswap dedup research](https://github.com/92Jay0810/zswap_dedup_research)

作者：

114065502 洪翊傑

114062625 李育鎧

2026

摘要

本研究延續課程 Initial Guide 階段一的研究問題：在多副本運算環境中，進入 zswap 的記憶體頁面（page）中，有多少比例的內容是重複的？此問題在容器（container）與虛擬機（VM）兩種虛擬化形式下，可能因記憶體共享機制、位址空間配置方式的不同，呈現不同的特性，因此本研究同時在兩種形式下進行量測，並額外開發自製壓力測試程式以排除既有測試工具的不可控變因。本研究由兩位成員分工進行：一人負責 eBPF 動態量測工具的開發，並以此完成容器端的實驗；另一人將該工具套用於 VM 端進行對應量測，並開發自製壓力測試程式以驗證結論的穩健性。

截至本版報告，容器端已透過自製的 eBPF 動態量測工具，掛載於 kernel 6.8 的 zswap_store() 函式進入點，在三類代表性容器化 workload（Media Streaming、Hotel Reservation、Python API Farm）、兩種 pod 規模、兩種 ASLR 設定下，完成 8 組獨立量測，累積超過 1,500 萬個 swap-out 事件樣本；後續又以組員自製的 host 端 C 語言壓力程式，針對 Media Streaming 容器 workload 補做純應用負載與外部亂數壓力源解耦的敏感度分析。

容器端核心發現：(1) 原始 8 組 stress-ng 量測的 dedup ratio 介於 16.16% 至 20.78% 之間，平均約 17.6%，遠高於既有文獻於 Android 單機場景量測的 5% 基準；(2) 使用 host 端不可壓縮亂數壓力源後，Media Streaming workload 在早期 swap 階段仍可量得 12.45%–13.19% 的 dedup ratio，但在深度壓力下下降至 5.57% 與 2.59%，說明容器端去重潛力主要出現在早期冷頁面回收階段，且會受壓力來源與 swap 深度影響；(3) 原始量測中重複頁面以 PTR_HEAVY（heap/stack 結構）與 TEXT/ASCII（容器平台 metadata、監控資訊）為主，補充量測則進一步凸顯 ZERO、SAME_FILLED 與 TEXT/ASCII 在外部亂數壓力模型下的比例變化。

本研究完成 VM 環境之 Host-level zswap deduplication 量測平台建置，包含 KVM 多虛擬機環境、Guest ASLR 控制、自製記憶體壓力程式以及 eBPF 量測工具，建立一套可重複且可控制的實驗流程。透過不同 Guest ASLR 組合之量測結果可發現，Host 端皆可觀察到一定程度的 page deduplication 現象，但整體 dedup ratio 並未因 ASLR 開關呈現明顯的單向變化。

進一步分析 Page Type Distribution 後發現，目前重複頁面主要來自 Guest OS 所產生之 ZERO page，而 PTR_HEAVY、BINARY 與 TEXT/ASCII 等頁面則占較小比例，顯示 VM 環境中的 deduplication 行為除了受到應用程式 workload 影響外，也與 Guest OS 記憶體管理機制及 Host 端 memory reclaim policy 密切相關。因此，Host-level dedup ratio 並不足以完全反映應用程式本身之 page similarity，後續仍需進一步區分系統頁面與應用程式頁面，以提升分析結果的代表性。

此外，為避免既有壓力測試工具可能產生固定內容 page 而影響量測結果，本研究自行開發記憶體壓力程式取代 stress-ng。實驗結果顯示，自製工具能有效降低 SAME_FILLED page 的比例，提供較可控制且可重現的記憶體壓力來源，使量測結果更能反映 VM 環境實際的 deduplication 行為，並降低壓力工具本身對實驗結果造成的干擾。

綜合而言，本研究完成 VM 平台之 deduplication 量測方法與實驗流程驗證，並初步釐清 Guest ASLR、系統頁面及 workload 對 Host-level zswap deduplication 的影響，作為後續建立更完整 VM 記憶體重複率分析與跨虛擬化平台比較之基礎。

目錄

摘要	2
目錄	3
1. 問題背景	5
2. Research Questions	6
3. 方法設計：容器端 eBPF 量測工具	7
3.1 Hook 點的選擇	7
3.2 從 folio 取得頁面內容：KASLR 動態常數問題	7
3.3 Hash 演算法：FNV-1a 取代 xxHash64	7
3.4 擴充設計：Thrashing 偵測與頁面內容分類	8
3.4.1 Memory thrashing 偵測	8
3.4.2 頁面內容分類	8
3.5 BPF Verifier Instruction Count 限制（kernel 6.8 特有問題）	9
4. 實驗設計：容器端	10
4.1 實驗假設與環境現象的對應	10
4.2 Workload 來源與選擇合理性	10
4.2.1 Benchmark 來源與學術依據	10
4.2.2 三個 Workload 的選擇理由	11
4.3 Workload 參數矩陣	12
4.4 環境設定	12
4.5 參數選擇依據	12
4.6 統計顯著性與資料品質確保	13
5. 量測結果：容器端	14
5.1 總體 Dedup Ratio	14
5.2 Workload 間差異	15
5.3 Pod 規模效應	16
5.4 重複頁面內容類型分布	16
5.5 Sharing Factor 分布	17
6. 討論：跨 Workload 一致的全零頁面 Thrashing 現象	19
6.1 現象描述	19
6.2 可能的成因分析	19
6.3 ASLR 對 Dedup Ratio 的影響	20
7. 方法設計與實驗結果：VM 端	22
7.1 已完成的環境基礎	22
7.2 方法設計	22
7.3 實驗設計	23
7.3.1 實驗假設	23
7.3.2 Workload 來源與合理性	23
7.3.3 環境設定與參數選擇	23
7.3.4 統計顯著性確保	24
8. 方法設計與實驗結果：自製壓力測試程式	25
8.1 動機：為何需要自製壓力測試工具	25
8.1.1 容器端補充量測：host 端亂數壓力源敏感度分析	25
8.2 方法設計	26

8.4 實驗結果	27
9. 研究限制與後續方向	28
9.1 量測工具的固有限制	28
9.1.1 eBPF 動態量測工具	28
9.1.2 自製壓力測試工具	28
9.2 記憶體區段對應與動態追蹤：跨虛擬化形式的未完成項目	29
9.3 VM 端 workload 完善	29
9.4 後續研究方向	30
10. 結論	31

1. 問題背景

Linux 的 zswap 是壓縮形式的 swap cache：當匿名頁面（anonymous page）因記憶體壓力被 swap out 時，zswap 會先將其壓縮並存放於記憶體中的 zsmalloc pool，避免緩慢的磁碟 I/O。然而，zswap 目前的實作不會檢查頁面內容是否與已儲存的其他頁面重複——若 N 個內容相同的頁面進入 zswap，系統會執行 N 次獨立壓縮、配置 N 份儲存空間、建立 N 個 zswap_entry，造成 CPU 與記憶體的雙重浪費。

這個浪費在多副本運算環境（多容器、多虛擬機）中特別顯著：多個 instance 共享相同的 base image、相同的 runtime（如 glibc、JVM、Python interpreter）、相同的 orchestration agent，在記憶體壓力下 swap out 時，自然容易產生大量內容重複的頁面。Kernel Same-page Merging（KSM）雖然能在記憶體充裕時合併相同頁面，但其作用範圍止於 swap-out 之前；一旦頁面進入 swap 層，KSM 的合併關係即告斷裂，因為 swap-out 通常發生在記憶體壓力最大、KSM 最沒有餘裕運作的時刻。

過去文獻對 swap 層級的內容重複率量測多侷限於單機場景：既有研究針對 zram 提出的 dedup patch，在 Android 單機裝置上僅量得約 5% 的重複率，因效益不足而未被 mainline 採納。然而，容器化與虛擬化多副本場景下的重複率從未被系統性量測過，這正是本課程 Initial Guide 階段一所設定的核心研究問題。

Initial Guide 為此問題設計了明確的決策流程：若量測得到的 dedup ratio 超過 10%，代表 content dedup 有足夠的收益空間，值得進入階段二的 kernel 修改；若所有 workload 皆低於 5%（與 Android 單機結果一致），則 zswap 層級的 content dedup 在工程上不具效益。本研究即依此流程，分別在容器與 VM 兩種虛擬化形式下進行量測，並額外開發自製壓力測試工具以排除既有測試工具（stress-ng）的不可控變因。

（完整問題定義與既有文獻請參照課程 Initial Guide 文件，此處不再重複展開。）

2. Research Questions

本研究圍繞以下 4 個研究問題展開。RQ1–RQ3 適用於容器與 VM 兩種虛擬化形式——同一組問題在兩種環境下各自量測一次，其結果的對照本身即回答了「容器與 VM 是否有系統性差異」這個問題，因此不另立比較型的研究問題。RQ4 則透過自製壓力測試工具，檢驗 RQ1–RQ3 的結論在排除既有測試工具不可控變因後是否仍然穩健。

編號	問題
RQ1	在多副本運算環境（容器或虛擬機）下，進入 zswap 的記憶體頁面中，有多少比例的內容是重複的？此重複率是否顯著高於既有文獻量測的單機基準（Android 單機 ~5%）？不同虛擬化形式（容器 vs VM）之間，量測到的重複率是否有系統性差異？
RQ2	若重複率顯著，這些重複頁面的內容來源是什麼——是應用層資料的巧合重複，還是執行環境（runtime、平台層 metadata，或 VM 的 guest OS 結構）本身結構性產生的重複？
RQ3	ASLR 是否會透過隨機化頁面中嵌入的指標值，降低可量測到的內容重複率？此效應的量級相對於 workload 類型本身的差異有多大？此效應在容器與 VM 兩種虛擬化形式下是否一致——例如 VM 因 guest kernel 可能採用較固定的位址映射，受 ASLR 影響的程度是否確實小於容器？
RQ4	排除 stress-ng 的不可控變因後，自訂的壓力寫入模式與多 workload 啟動時間差，是否會改變量測到的 dedup ratio？這能否驗證或推翻 RQ1–RQ3 在受控條件下的結論？（待組員依自製壓力測試實際進展確認／調整文字）

本研究由兩位成員分工進行：一人開發 eBPF 動態量測工具並完成容器端對 RQ1–RQ3 的量測（見第 3–6 章）；另一人將該 eBPF 工具套用於 VM 端，完成 RQ1–RQ3 在虛擬機場景下的對應量測（見第 7 章），並開發自製壓力測試工具負責 RQ4（見第 8 章）。兩條路徑共用同一套 eBPF 量測工具，確保容器與 VM 的觀測指標、資料格式完全一致，便於直接比較。三者共同構成 Initial Guide 階段一決策流程所需的完整證據基礎：若容器與 VM 場景的量測一致支持 dedup ratio 顯著高於 10% 門檻，且此結論在排除 stress-ng 變因後仍然穩健，則可確認進入階段二 kernel 修改具有足夠的工程價值。

3. 方法設計：容器端 eBPF 量測工具

本章說明容器端用於回答 RQ1–RQ3 的 eBPF 動態量測工具設計。VM 端採用相同的研究問題（RQ1–RQ3），其方法設計由組員另行撰寫，詳見第 7 章；自製壓力測試工具對應 RQ4，詳見第 8 章。

3.1 Hook 點的選擇

Initial Guide 原始設計建議將量測程式掛載於 `zswap_store_page()`（`mm/zswap.c` 約第 1402 行），理由是如此處為頁面進入 `zswap` 壓縮路徑的入口，且 `swap` 層的 `zeromap` 機制已在更上層過濾掉全零頁面，因此抵達此處的頁面語意上保證為非全零內容。

然而，本研究的量測環境為 `kernel 6.8`（`Ubuntu 24.04`），該版本已完成 `folio` 資料結構遷移，`zswap_store_page()` 函式已不存在於 `kallsyms` 中。透過 BTF（BPF Type Format）確認，實際入口函式為 `zswap_store(struct folio *folio)`，其參數型別由 `struct page *` 變更為 `struct folio *`。此外，BTF 顯示該函式為 `static linkage`，部分 `kernel` 設定下 `fentry` 無法掛載 `static` 函式，因此最終改用 `kprobe` 掛載：

```
SEC("kprobe/zswap_store") int BPF_KPROBE(zswap_store_entry, struct folio *folio)
```

此 hook 點與 Initial Guide 的設計語意等價：每個通過 `zeromap`、進入 `zswap` 的非零頁面都會觸發此 hook。

3.2 從 folio 取得頁面內容：KASLR 動態常數問題

`kprobe` 在 `zswap_store()` 處取得的是 `struct folio *`，但量測頁面內容需要取得該頁面在 `kernel space` 的虛擬位址（`virtual address`），這需要透過以下推算：

```
pfn = (folio_addr - vmemmap_base) / sizeof(struct page) // 64 bytes on x86-64 virt =
page_offset_base + pfn * PAGE_SIZE
```

此推算所需的兩個常數 `vmemmap_base` 與 `page_offset_base` 在 `kernel 6.8` 環境下是執行期動態變數，因 KASLR（Kernel Address Space Layout Randomization）而在每次開機後產生不同偏移，無法以教科書預設值硬編碼（實測硬編碼會導致全部頁面讀取失敗，`error rate` 達 100%）。本研究亦嘗試透過 `/proc/kcore` 讀取，但發現該 `kernel` 版本的 `kcore` 並未提供任何 `PT_LOAD segment`，此路徑不可行。

最終解法：`userspace` 程式從 `/proc/kallsyms` 讀取兩個變數的符號位址（非數值），透過 BPF `map` 注入 BPF 程式；BPF 程式在第一次觸發時使用 `bpf_probe_read_kernel()` 讀出實際數值並快取於 `map` 中，後續直接使用快取值。此設計完全不依賴硬編碼常數，重開機後可自動適應新的 KASLR 偏移。

3.3 Hash 演算法：FNV-1a 取代 xxHash64

Initial Guide 建議使用 `kernel` 內建的 `xxHash64` 作為內容雜湊演算法。然而 `xxHash64` 使用 4 個 `accumulator lane` 並行運算，搭配 `#pragma unroll` 展開迴圈後，`clang` 編譯器會將大量中間變數保留在 `stack` 上，超過 BPF 程式 512 bytes 的 `stack` 上限。嘗試以 `__noinline` 拆分函式或調整實作皆未能解決此問題。

本研究改採 FNV-1a 64-bit 演算法，其每一步運算僅需 XOR 與乘法兩個操作，所需暫存空間僅一個 u64，搭配 kernel 5.17 引入的 `bpf_loop()` API（取代 `#pragma unroll`），verifier 僅需驗證 callback 函式本身，無需展開整個迴圈，徹底消除了 stack 爆炸問題。對於本研究的去重量測用途（非密碼學應用），FNV-1a 的碰撞率足夠低。頁面內容（4096 bytes）存放於 `BPF_MAP_TYPE_PERCPU_ARRAY`，以繞過 BPF stack 限制並避免跨 CPU 鎖競爭。

3.4 擴充設計：Thrashing 偵測與頁面內容分類

在初步量測確認 dedup ratio 具有研究價值後，量測工具擴充了兩項分析能力，用於支援 RQ2（重複內容歸因）：

3.4.1 Memory thrashing 偵測

量測工具為每個內容 hash 維護 `first_seen_ns` 與 `last_seen_ns` 兩個時間戳，每次同一 hash 再次出現時計算 interval（與上次出現的時間差）。interval 小於 5 秒的重複事件被標記為 thrashing：代表同一份內容在短時間內被反覆 swap in/out，是記憶體壓力下系統「左腳踩右腳」現象的直接證據。

3.4.2 頁面內容分類

為了在不取得 user-space virtual address 的前提下推斷重複頁面的資料性質，工具對每個重複頁面在 4 個等距位置（offset 0、1024、2048、3072，各取 32 bytes）取樣，送至 userspace 由啟發式規則分類：

- ZERO：4 個取樣位置全部為 0x00（真正全零頁面，理論上應已被 zeromap 過濾）
- SAME_FILLED：4 個位置全部為同一非零 byte 值
- TEXT/ASCII：可列印字元比例 $\geq 70\%$ （應用層字串、JSON、設定檔等）
- CODE(x86)：符合 x86-64 function prologue pattern（如 push rbp、endbr64）
- PTR_HEAVY：取樣中 ≥ 3 個 8-byte 區塊符合 canonical 64-bit pointer 格式（heap/stack 結構特徵）
- BINARY：以上皆不符合的其他二進位資料

分類器最初僅檢視頁面開頭 64 bytes，導致部分「開頭恰好為零、但頁面其餘部分有內容」的頁面被誤判為 ZERO。修正後的版本要求 4 個分散取樣位置皆確認為零才判定為真正的 ZERO，並在事件中附加 `is_zero_confirmed` 欄位供驗證。

3.5 BPF Verifier Instruction Count 限制 (kernel 6.8 特有問題)

量測工具在擴充過程中曾於開發環境完成初版實作，但在載入目標環境 kernel 6.8 時失敗，verifier 回報：

```
BPF program is too large. Processed 1000001 insn processed 1000001 insns (limit 1000000)
```

kernel 6.8 的 verifier instruction 上限為 100 萬，較新版本更為嚴格。問題根源是頁面分類邏輯使用了 4 個獨立的 bpf_loop() callback (零值檢查、same-filled 檢查、ASCII 檢查、pointer 檢查)，加上原有的 FNV-1a hash callback，共 5 個 callback。kernel 6.8 的 verifier 對每個 bpf_loop callback 的 state 追蹤成本顯著高於後續版本，五者合計超過上限。將分類函式由 __always_inline 改為 __noinline 並未解決問題，因為瓶頸並非函式展開本身，而是 callback 數量過多導致的 verifier state 爆炸。

最終解法：將整個頁面分類邏輯移至 userspace 執行。BPF 端僅保留一個 bpf_loop() callback (FNV-1a hash)，其餘工作 (讀取頁面、計數、送出 ringbuf 事件) 皆為線性程式碼，不含迴圈；分類邏輯與統計改在 userspace 的 C 程式中以一般迴圈完成，不受 BPF verifier 限制。代價是頁面類型統計從「每個頁面」退化為「僅統計觸發 ringbuf 事件的重複頁面」，但對本研究的目的 (理解重複頁面的內容組成) 而言已足夠。此架構調整使量測工具能在 kernel 6.8 上穩定運作，為後續正式量測的基礎。

4. 實驗設計：容器端

4.1 實驗假設與環境現象的對應

本實驗的核心假設是：容器化多副本環境中，因多個 pod 共享相同 base image、相同 runtime（interpreter、shared library 或靜態打包 binary）與相同 orchestration agent，其記憶體配置會在 swap 層級產生顯著高於單機場景的內容重複。為了讓量測結果能夠回應這個假設，實驗設計需要刻意納入以下幾個真實容器環境中會發生、且可能左右量測結果的現象：

- **Page cache 與 anonymous page 的區別：**單純的網路壓測（例如以 ab 對串流服務發送請求）主要產生 file-backed page cache，這類頁面在記憶體不足時會被系統直接捨棄，不會經過 swap-out，因此無法觸發 zswap。實驗刻意納入 stress-ng --vm 對每個 pod 鎖定期固定大小的記憶體，強制配置無法被直接捨棄的 anonymous page，才能逼迫 kernel 將其他閒置 pod 的 stack、heap 等匿名記憶體一併推入 zswap，這是觀測到 content dedup 效果的必要條件。
- **LRU 深度與 base image 共用頁面的關係：**容器的記憶體頁面在 LRU（least-recently-used）佇列中有新舊分層——佇列頂層多是高度變動、由壓力測試工具本身產生的隨機資料；佇列底層則可能包含各容器在啟動初期、長期未被改寫的 base image 初始化結構與環境變數頁面。只有當記憶體壓力深入到驅使 kernel 回收 LRU 底層頁面時，才會看到 base layer 的共用內容被推入 zswap。這意味著實驗必須確保壓力測試持續足夠長的時間、且強度足以觸及 LRU 深層，否則量測到的 dedup ratio 會被淺層回收的高變動性頁面拉低，低估容器環境真實的重複潛力。
- **ASLR 在高壓場景下的角色：**理論上 ASLR 會隨機化嵌入頁面中的指標值（stack frame、heap metadata），降低內容重複率；但在極端記憶體壓力下，被擠入 zswap 的頁面組成可能以純資料頁面（data / buffer）為主，包含指標、受 ASLR 影響的 metadata 頁面（如 stack frame）占比可能遠低於預期。實驗設計納入 ASLR 開／關的對照組，目的即是驗證這個假設在受控條件下是否成立。

4.2 Workload 來源與選擇合理性

容器端的 workload 篩選標準是「Runtime & Memory Layout Characteristics」，即根據技術棧在 shared library 依賴性、記憶體配置器（allocator）行為、以及資料集中度上的差異，挑選三個彼此形成對比、且在這些維度上具有代表性的應用，以求單一組實驗矩陣能涵蓋現代雲端應用的主要記憶體特徵組合，而非僅針對單一技術棧得出結論。

4.2.1 Benchmark 來源與學術依據

三個 workload 中，Media Streaming 與 Hotel Reservation 分別取材自學界廣泛採用、具有同行評審文獻支持的標準化 benchmark 套件，而非自行拼裝的測試腳本，這確保了實驗的負載特徵具備外部效度，量測結果可與其他採用相同 benchmark 的研究相互比較。

Media Streaming (CloudSuite)：取自 CloudSuite Benchmark Suite，由瑞士洛桑聯邦理工學院（EPFL）開發，是學界公認權威的開源資料中心基準測試集，專門用於模擬資料中心中真實的規模化（scale-out）workload，涵蓋大數據分析、網頁服務、媒體串流等場景。CloudSuite 的設計目標即是讓研究者能在受控環境中重現雲端資料中心的典型負載特徵，因此被廣泛用於電腦架構與系統研究中作為標準測試基準。本研究採用其 Media Streaming 場景，對應論文：Ferdman

et al., "Clearing the Clouds: A Study of Emerging Scale-out Workloads on Modern Hardware," ASPLOS 2012 (<https://dl.acm.org/doi/10.1145/2150976.2150982>)。

Hotel Reservation (DeathStarBench)：取自 DeathStarBench，由美國康乃爾大學（Cornell University）開發的微服務 benchmark 測試套件。DeathStarBench 的特色是提供多種貼近真實世界的端到端微服務應用拓模，包含社交網路（Social Network）、媒體服務（Media Service）、電子商務網站（E-commerce）、銀行系統（Banking System），以及用於無人機群協同控制（Swarm of Drones）等複雜 workload，每個應用皆由數十個獨立、以不同程式語言實作的微服務組成，service-to-service 通訊則透過標準的 RPC 框架（如 Thrift、gRPC）完成。本研究採用其 Hotel Reservation 應用（屬社交網路與服務導向架構的範疇，模擬旅館訂房系統的多微服務協作），對應論文：Gan et al., "An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems," ASPLOS 2019 (<https://dl.acm.org/doi/10.1145/3297858.3304013>)。選用 DeathStarBench 而非自行撰寫微服務測試腳本的理由是：真實微服務拓模中服務間呼叫鏈的深度與並發模式，會直接影響各服務 pod 的記憶體配置時序與內容特徵，這是本研究第 4.1 節欲納入的真實環境現象之一，自行撰寫的簡化腳本難以重現這種複雜度。

Python API Farm：為配合 4.2 節「Runtime & Memory Layout Characteristics」篩選標準中，針對動態連結 / interpreter 型 runtime 的對照需求而設計的自訂 workload，模擬多個 Python Web API 服務副本共享同一 interpreter 與系統層級 shared library 的場景，並非取自上述兩個標準 benchmark 套件。

4.2.2 三個 Workload 的選擇理由

在篩選過程中，本研究亦排除了不適合納入容器端 workload 矩陣的候選——**CloudSuite Benchmark 的 Spark In-Memory Analytics**。排除理由有三：(1) JVM 在記憶體進入 swap 後容易引發嚴重的 GC thrashing，垃圾回收機制需定期掃描全域記憶體，一旦 heap 區段進入 swap，會強制將冷資料頻繁讀回實體記憶體，在實測中導致 pod 頻繁逾時並遭 Kubernetes 強制終止，無法維持穩定的長時間壓測環境；(2) Spark 產生大量具高度隨機性、唯一性的 RDD 與 Java 物件，這些物件進入 swap 後重複率天然極低，無法代表一般微服務的真實共享狀況，反而會稀釋實驗訊號；(3) Spark 本質上是單一進程、極大工作集的重型資料分析應用，與本實驗矩陣所需的「高數量、輕量級、同質性」容器設定在工作負載定位上不相容。

最終選定的三個 workload 及其選擇理由如下：

Workload	技術棧特徵	驗證目標	預期 ASLR 敏感度
Python API Farm (動態連結 / Interpreter)	多 pod 共享同一 interpreter 與系統層級 shared library (如 libpython3.so)	OS 層級 shared library 的 dedup 效率極限；預期 pod 數增加帶動 dedup ratio 線性成長	高 (關閉 ASLR 可測得理論上限)
Media Streaming (高效能單體, C++)	I/O-intensive, 處理大量併發資料流, 記憶體配置集中於少數大型 buffer	極限 swap 壓力下的系統延遲 baseline；預期 sharing factor 呈雙峰分布 (系統結構體高重複、串流資料近乎不可去重)	低 (多為 data/buffer page)
Hotel Reservation (靜態編譯微服務, Go)	所有依賴與 runtime 打包為單一 binary, 多個小型微服務協同運作	靜態打包技術在傳統 4KB-granularity dedup 機制下的效益與瓶頸	中 (主要影響 data segment)

表 4-1 容器端 Workload 選擇理由與預期觀測重點

4.3 Workload 參數矩陣

各 workload 的 pod 規模、stress-ng 壓力參數、單 pod 記憶體配額如下表。記憶體配額的設定原則是讓叢集總壓力維持在相近區間（約 5–7.2 GB），使跨 workload 比較時，總記憶體壓力不是造成 dedup ratio 差異的混淆變因。

負載名稱	Pod 數	stress-ng 參數	Mem Req	Mem Lim	預估總壓力
W2-A: Media Streaming (C++)	10	--vm-bytes 800M	400 Mi	1 Gi	~7.2 GB
W2-A: Media Streaming (C++)	30	--vm-bytes 240M	80 Mi	300 Mi	~6.9 GB
W2-B: Hotel Reservation (Go)	12	--vm-bytes 600M	300 Mi	700 Mi	~6.0 GB
W2-C: Python API Farm (Py)	10	--vm-bytes 500M	200 Mi	650 Mi	~5.0 GB

表 4-2 Container Workload 參數矩陣

Media Streaming workload 額外測試 10 pods 與 30 pods 兩種規模：30 pods 設定下單一 pod 的記憶體配額大幅縮小（請求從 400 Mi 降至 80 Mi，上限從 1 Gi 降至 300 Mi），但叢集總壓力維持相近，使得「pod 數量」與「總記憶體壓力」兩個變因得以解耦，可單獨觀察 pod 數量本身（而非總壓力大小）對 dedup ratio 的影響。

4.4 環境設定

- Kubernetes 發行版：k3s（輕量化單節點叢集），所有 workload 透過 kubectl apply 部署
- Kernel 版本：Linux 6.8（Ubuntu 24.04），zswap 透過 `/sys/module/zswap/parameters/enabled` 確認啟用，並掛載 `debugfs` 以讀取 `/sys/kernel/debug/zswap/` 下的核心計數器作為輔助驗證
- ASLR 控制：透過 `/proc/sys/kernel/randomize_va_space` 切換（0 = 關閉，2 = 系統預設完整隨機化），每組 workload 皆於兩種設定下各量測一次
- 記憶體壓力產生：以 `stress-ng --vm-bytes` 對每個 pod 鎖定固定大小的匿名記憶體，迫使系統面臨無法透過捨棄 `page cache` 緩解的記憶體壓力
- 量測工具：第 3 章所述的 eBPF kprobe 動態追蹤工具，以 60 秒為統計輸出間隔，每組量測持續至 swap 流量穩定且累積足量樣本

4.5 參數選擇依據

壓力測試參數（`stress-ng --vm-bytes`）的數值選擇，是先以 `top` 指令觀察系統的 MiB Swap 區塊：當 `stress-ng` 開始發力時，實體記憶體的 `free` 會急速下降，隨後 `swap` 的 `used` 數值開始攀升，這個轉折點代表系統已進入高壓換頁狀態，正是擷取 eBPF 數據的最佳時機。各 workload 的 `--vm-bytes` 數值即依此原則，反覆測試調校至能穩定觸發大量 `swap-out`、且不會因壓力過大導致 pod 被 Kubernetes 強制終止的區間。

Pod 的記憶體請求（request）與上限（limit）設定，依循 Kubernetes 的資源排程機制：request 影響排程器是否能將 pod 分配到節點，limit 則是 cgroup 層級的硬性記憶體上限，超過上限會觸發 OOM kill。本實驗透過調整 limit 接近但略高於應用程式的正常工作集大小，確保記憶體壓力主要由 stress-ng 主動製造，而非應用程式本身的不穩定記憶體使用造成。

4.6 統計顯著性與資料品質確保

為確保不同 workload 之間的量測數據具備可比較性，並排除 hash collision 對 dedup ratio 數字的污染，本實驗採用以下標準流程：

- 環境初始化一致性：每組量測前清除現有 swap、重新確認 ASLR 與 zswap 系統變數，確保不同組別的起始狀態一致，避免前一組殘留的 swap 內容干擾下一組的統計
- Byte-for-byte 抽樣驗證：對於 hash 命中（即判定為重複）的頁面，抽樣進行逐位元組比對，確認內容確實完全相同而非 FNV-1a 64-bit 的雜湊碰撞。在本研究的樣本規模下（單組量測約 100–300 萬頁面），64-bit 雜湊空間的碰撞機率在理論上極低，抽樣驗證的目的是提供經驗證據而非僅依賴理論機率
- 讀取錯誤率作為資料品質門檻：每組量測皆檢查 read error rate（KASLR 常數讀取失敗或頁面讀取失敗的比例），全部 8 組量測的 error rate 皆為 0%，確認資料完整性後才納入正式分析
- 每組量測的樣本量門檻：各組量測皆持續至累積超過 100 萬筆 swap-out 事件以上才停止收集，避免樣本量過小導致的統計波動被誤判為 workload 間的真實差異
- 重複量測的交叉驗證：同一 workload 在 ASLR 開／關兩種條件下各量測一次，形成自然的對照組；若兩者的 dedup ratio 差異遠小於 workload 間的差異，可作為「workload 類型才是主要影響因素」此一結論的佐證（詳見第 5 章）

5. 量測結果：容器端

本章依序呈現 8 組量測的總體結果，並在每一節明確標示第 4.1 節提出的預期與實際觀測結果的對照，以利檢視假設是否成立、以及量測過程是否存在需要進一步檢查的異常。

5.1 總體 Dedup Ratio

8 組量測的最終統計彙整如下表。所有量測的讀取錯誤率（read error rate）皆為 0%，確認 KASLR 動態常數機制在 kernel 6.8 環境下運作正確。

Workload	Pods	ASLR	Total pages	Dedup ratio	Thrashing events	Mem saved (MB)
Media Streaming (C++)	10	off	1,167,861	16.39%	133,830	747.8
Media Streaming (C++)	10	on	1,214,526	16.75%	137,100	794.6
Media Streaming (C++)	30	off	1,327,382	17.85%	147,666	925.7
Media Streaming (C++)	30	on	1,954,147	17.59%	223,566	1342.6
Hotel Reservation (Go)	12	off	3,239,710	20.78%	342,888	2629.8
Hotel Reservation (Go)	12	on	2,109,520	18.93%	246,500	1559.8
Python API Farm (Py)	10	off	2,499,151	16.54%	220,162	1614.5
Python API Farm (Py)	10	on	1,859,955	16.16%	166,015	1173.9

表 5-1 8 組量測最終統計彙整

【對應 RQ1，預期：符合】8 組量測的 dedup ratio 介於 **16.16%** 至 **20.78%** 之間，平均約 17.6%。對照 Initial Guide 設定的決策門檻——dedup ratio 超過 10% 即代表 content dedup 有足夠收益空間值得進入階段二——全部 8 組量測結果皆穩定超過此門檻一倍以上，且全部超過既有文獻於 Android 單機場景量測到的 5% 基準三倍以上。這與第 4.1 節的核心假設一致：容器化多副本環境因共享 base image、runtime 與 orchestration agent，產生的 swap 內容重複率顯著高於單機場景，RQ1 得到肯定的答案。

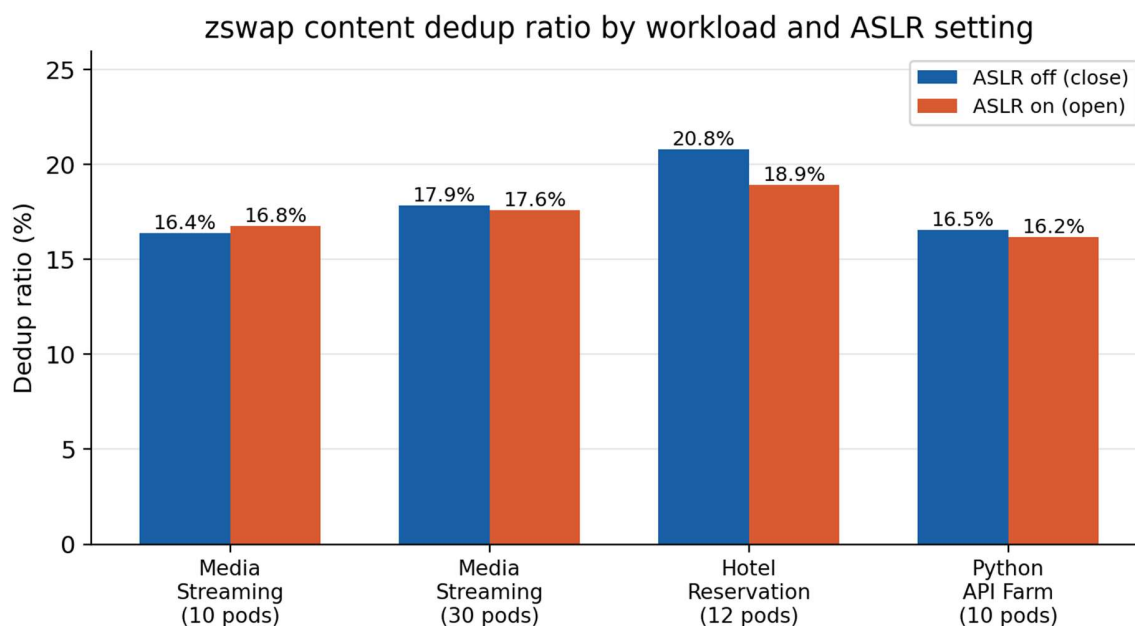


圖 5-1 zswap 內容去重率：跨 workload 與 ASLR 設定比較

5.2 Workload 間差異

【對應 RQ2，預期：部分不符，需檢討】第 4.1 節依「Runtime & Memory Layout Characteristics」標準提出的預期是：Python API Farm 因多 pod 共享同一個 interpreter 與系統層級 shared library（如 libpython3.so），應展現最高的 OS 層級 dedup 效率；Hotel Reservation 因採靜態編譯（所有依賴打包進單一 Go binary，不依賴外部 shared library），理論上 dedup 空間應較受限。

然而實測結果方向相反：在 ASLR off 條件下比較三類 workload（以相近 pod 數量為基準），**Hotel Reservation (Go, 12 pods)** 的 dedup ratio 最高，達 **20.78%**，高於 Media Streaming (C++, 10 pods, 16.39%) 與 Python API Farm (10 pods, 16.54%) ——Python API Farm 並未如預期展現最高的重複率。

這個落差有兩個可能的解讀方向，皆需要後續實驗才能確認：第一，shared library 本身雖然在磁碟上是同一份檔案，但 mmap 進各 pod 行程後，library 的 .text（程式碼）段在正常運作下多為 file-backed page，會被頁面快取機制直接共享、且不容易被 swap-out 成 anonymous page，因此本實驗設計的 stress-ng 壓力測試路徑，可能主要捕捉到的是 Python 應用層的 heap 物件（直譯器的物件配置天然具有高度多樣性），而非真正測到 shared library 的 dedup 上限；第二，Hotel Reservation 是由多個小型微服務（frontend、profile、reservation、geo 等）共同組成，服務數量與程序間通訊頻率皆高於單一大型 C++ 串流程式或單一 Python API 服務，Go runtime 自身的記憶體配置模式（goroutine stack、GC heap 結構）在多個相同二進位檔的副本之間，可能比預期更具規律性、更容易產生重複頁面。

這個觀察提示了實驗設計上一個值得留意的限制：本研究的「驗證目標」設計是針對單一技術棧的理論特性（如 shared library 共享），但實際量得的 dedup ratio 是該技術棧在特定壓力測試手法下、整個記憶體子系統（runtime + 應用層配置 + 微服務拓樸）共同作用的結果，三者之間的

因果關係尚未透過控制變因實驗（例如固定微服務數量、只改變 runtime 語言）拆解開來，這是本研究結論中需要明確標註不確定性、留待後續驗證的部分。

5.3 Pod 規模效應

【預期：符合】比較 Media Streaming workload 在 10 pods 與 30 pods 下的結果，**dedup ratio** 隨 pod 數量增加而上升：ASLR off 時由 16.39% 升至 17.85%（+1.46 個百分點），ASLR on 時由 16.75% 升至 17.59%（+0.84 個百分點），與第 4.1 節「pod 數量增加將帶動 dedup ratio 成長」的預期方向一致。值得注意的是，30 pods 設定下單一 pod 的記憶體配額被大幅縮小（記憶體請求從 400 Mi 降至 80 Mi，上限從 1 Gi 降至 300 Mi），但叢集總壓力維持相近（約 6.9–7.2 GB），這代表 pod 數量增加帶來的 dedup ratio 提升，並非單純來自總記憶體壓力的變化，而更可能來自「更多獨立副本共享相同 base image / runtime」這一機制本身。

這個現象可以用 LRU 深度與 base image 共用頁面的關係（見 4.1 節）來解釋：當 swap 總頁數（total pages）較少時，kswapd 僅能換出 LRU 佇列頂層、高度變動的頁面（壓力測試工具本身產生的隨機資料、network buffer 等），這些頁面在不同 pod 間天然不重複，量得的 dedup ratio 偏低；而當記憶體壓力持續推升 swap 總頁數，kernel 被迫深挖 LRU 佇列底層，此處包含各容器長期未被改寫的 base image 初始化結構與環境變數頁面——這些內容在所有共享同一 base image 的 pod 之間完全相同。30 pods 因總壓力與 10 pods 相近但副本數更多，更容易在相同的記憶體壓力下觸及這層深度回收區間，因此量得更高的 dedup ratio。換言之，pod 規模效應的本質可能不是「副本數量本身」線性貢獻重複率，而是「副本數量增加了 LRU 深層共用頁面被觸及的機率」。

此外，30 pods 設定下的 ZERO 類型頁面占比（35.1%–37.5%）顯著高於 10 pods 設定（20.1%–23.4%），這個現象在第 6 章有更詳細的討論。

5.4 重複頁面內容類型分布

【對應 RQ2】將每組量測中重複頁面（duplicate pages）依照 4 個取樣位置的內容特徵分類，結果如下：

Workload / ASLR	ZERO	PTR_HEAVY	TEXT/ASCII	BINARY	SAME_FILLED
Media-10 / off	23.4%	49.4%	20.5%	5.9%	0.8%
Media-10 / on	20.1%	58.5%	14.7%	5.8%	0.8%
Media-30 / off	35.1%	44.2%	14.9%	5.0%	0.8%
Media-30 / on	37.5%	41.5%	15.8%	4.6%	0.5%
Hotel-12 / off	14.3%	61.5%	18.1%	5.8%	0.3%
Hotel-12 / on	18.8%	56.9%	17.2%	6.6%	0.5%
Python-10 / off	22.3%	54.2%	16.1%	6.9%	0.5%
Python-10 / on	21.9%	56.8%	14.8%	5.8%	0.7%

表 5-2 重複頁面內容類型分布（僅統計重複頁面）

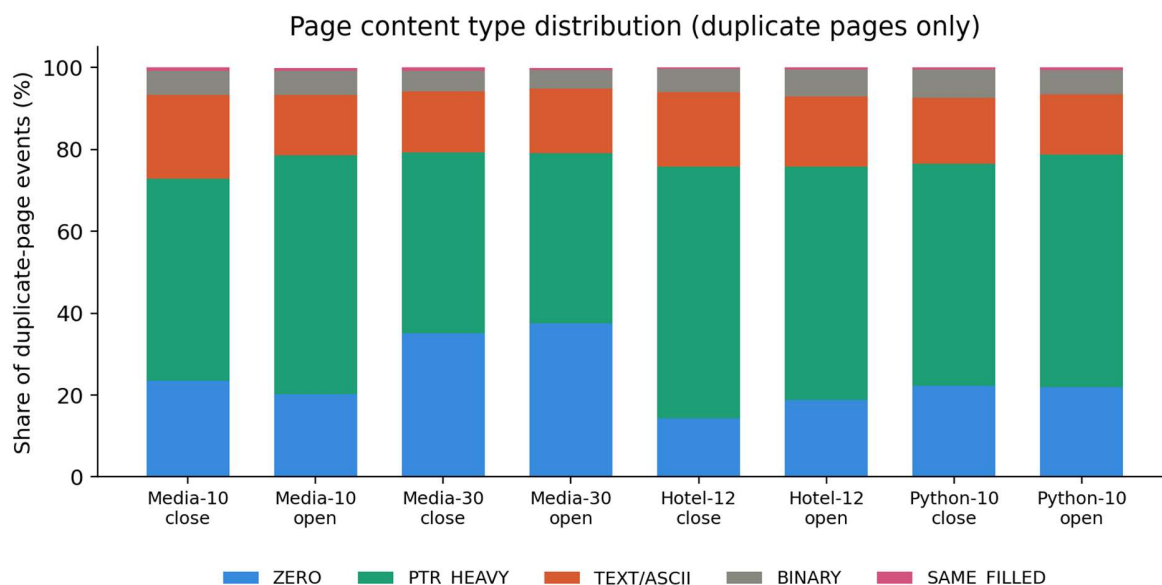


圖 5-2 重複頁面內容類型分布（堆疊比例圖）

跨全部 8 組量測，**PTR_HEAVY**（指標密集，對應 **heap/stack** 資料結構）穩定為占比最高的類型，介於 41.5% 至 61.5% 之間。這與容器化 workload 的記憶體配置特徵相符：大量 process（每個 pod 內的應用程式、k3s 元件、containerd shim 等）的 heap 與 stack 在配置模式相近時，會產生高度相似的指標填充模式（例如連續配置的 metadata 結構）。

TEXT/ASCII 類型占 14.7%–20.5%，內容抽樣顯示多為容器平台層級的 metadata 與監控資訊，包括：Kubernetes API Server 回傳的 JSON（如 `{"kind": "APIGroupList" ...}`）、cgroup 路徑字串（`/kubepods.slice/...`）、以及 `/proc/pressure` 的 PSI（Pressure Stall Information）輸出（`some avg10=0.00 ...`）。這些內容指向一個重要的重複來源：容器編排平台（k3s、containerd）與監控代理在多個 pod 間反覆產生格式相同、數值相近的狀態回報，這些回報暫存於 heap 上，在記憶體壓力下被 swap out 時自然形成大量重複內容。

5.5 Sharing Factor 分布

Sharing factor 描述「同一份內容被多少個獨立的 swap-out 事件共享」。下圖以 Media Streaming（10 pods）、Hotel Reservation、Python API Farm 三組（皆為 ASLR off）為例：

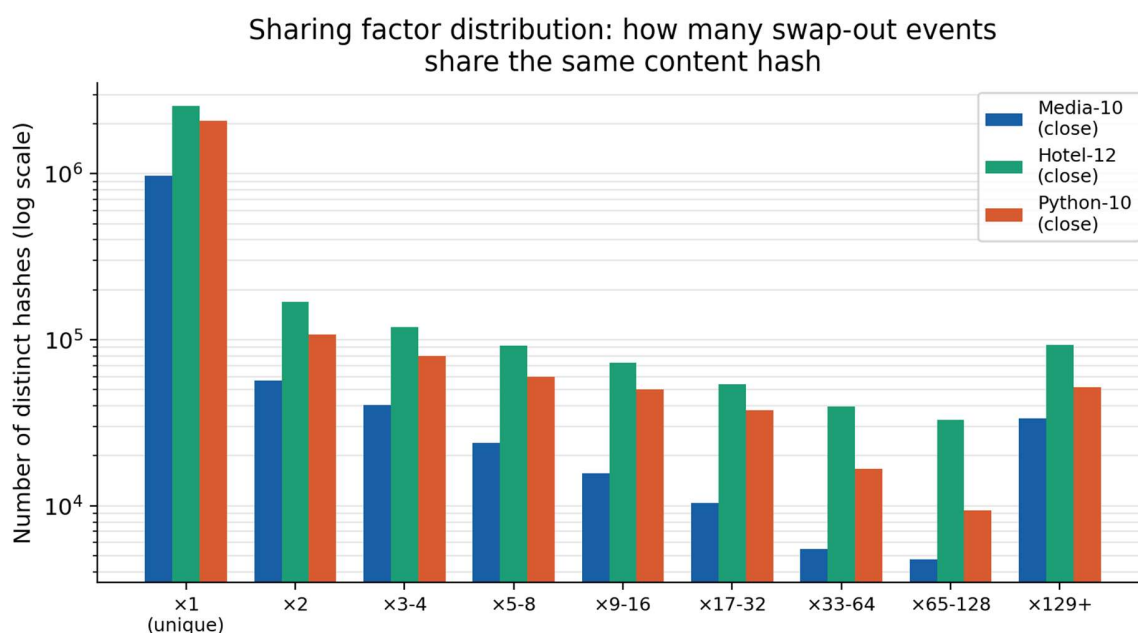


圖 5-3 Sharing factor 分布 (對數座標)

三組 workload 的分布形狀相近：多數內容雜湊只出現 1 次 (unique)，但右側 $\times 129+$ 區間的數量明顯高於中間區段 ($\times 33-64$ 、 $\times 65-128$)，形成一個「U 型」分布。這代表重複頁面的來源並非均勻分散在大量中等重複度的內容上，而是集中在少數被極端高頻共享的內容（如第 6 章討論的全零頁面與固定格式字串），疊加大量僅重複 2–8 次的偶發內容。這個分布形狀對於未來若進行階段二 kernel 修改、設計 dedup index 的資料結構，具有實際參考價值：少數高頻共享內容適合用簡單的 hash table 處理，無需為長尾的低頻內容過度設計。

6. 討論：跨 Workload 一致的全零頁面 Thrashing 現象

【對應 RQ2 的延伸發現】本章記錄一個在第 5 章資料分析過程中意外發現、且未被第 4.1 節原始假設預期到的現象——這個發現並非實驗設計時刻意要尋找的目標，而是在交叉比對 8 組量測的熱點雜湊時浮現的異常訊號，值得獨立成章詳細記錄。

6.1 現象描述

在分析每組量測中重複觸發次數最高的內容雜湊時，我們發現一個值得特別關注的現象：同一個內容雜湊（hash = 0x7da144b97d054b25，分類為 ZERO，即全零頁面）在全部 8 組獨立量測中，皆是觸發次數最高的雜湊，且該雜湊在 4 個取樣位置的零值確認比例（is_zero_confirmed）穩定在 99.9% 以上，排除了分類器誤判的可能性。

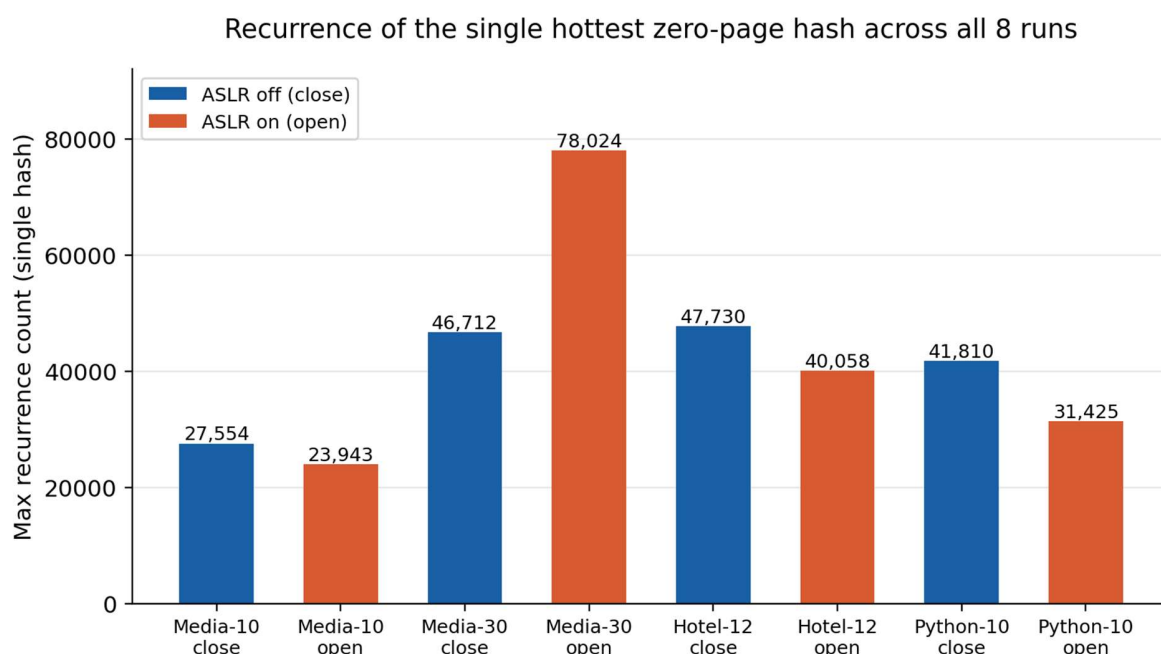


圖 6-1 單一全零頁面雜湊在 8 組量測中的最高重複觸發次數

此雜湊的觸發次數在不同 workload 與 ASLR 設定下分別為：

- Media Streaming 10 pods：27,554（ASLR off）／ 23,943（ASLR on）
- Media Streaming 30 pods：46,712（ASLR off）／ 78,024（ASLR on）
- Hotel Reservation 12 pods：47,730（ASLR off）／ 40,058（ASLR on）
- Python API Farm 10 pods：41,810（ASLR off）／ 31,425（ASLR on）

進一步計算此單一雜湊占對應量測中全部 thrashing 事件（interval < 5 秒）的比例，結果為 **13.9%（Hotel-12/off）至 34.9%（Media-30/on）**，意味著在某些量測條件下，超過三分之一的「短間隔重複 swap」事件，都來自同一個全零頁面反覆進出 zswap。

6.2 可能的成因分析

此現象具有兩個值得注意的特徵：第一，內容雜湊跨全部 8 組量測完全一致，代表這不是隨機產生的全零頁面，而是某個**系統性、可重現的記憶體配置 / 釋放模式**所產生；第二，`is_zero_confirmed` 比例接近 100%，理論上 swap 層的 zeromap 機制應該在 `swap_writeout()` 階段就攔截全零頁面，不應讓其進入 `zswap_store()`。

我們提出以下可能的成因假設，皆需要進一步的核心程式碼追蹤與驗證，本研究尚未對其逐一證實：

- **Large folio 邊界效應**：kernel 6.8 已支援多頁面的 large folio。若一個 large folio 中部分子頁面為全零、部分有內容，folio 整體仍會進入 `zswap_store()` 路徑，而 zeromap 的過濾邏輯可能僅在 single-page 路徑生效，導致 large folio 中的全零子頁面繞過過濾。
- **固定大小的核心或 runtime 緩衝區**：某個被頻繁配置與釋放的固定大小記憶體區塊（例如某 driver 的 DMA buffer、或某個被頻繁 mmap/munmap 的匿名映射）在初始化但尚未寫入內容前就被 swap out。
- **cgroup 或容器執行時的某個共同機制**：由於此雜湊在不同技術棧（C++、Go、Python）、不同 pod 規模下皆一致出現，且觸發來源主要為 `kswapd0`（佔比約 80–90%），暗示其來源可能與容器平台本身（k3s、containerd）或 host kernel 的共同基礎設施有關，而非應用層程式碼。

由於 `kprobe` 在 `zswap_store()` 處取得的程序情境（process context）是執行 swap-out 路徑的程序（多為 `kswapd`），而非頁面實際擁有者（page owner），本量測工具**無法直接確認此全零頁面屬於哪一個 process 的哪一段記憶體區域**——這正是 RQ2（重複內容歸因）在現有 hook 點下的根本限制，記憶體區段層級的精確對應需要額外的追蹤路徑才能完成（詳見第 9.2 節限制與後續方向）。確認此現象的根本原因，需要在 swap-in 路徑（`do_swap_page()`）額外掛載 `kprobe`，取得頁面擁有者的 VMA 資訊，這是本研究尚未完成、留待後續擴充的工作。

6.3 ASLR 對 Dedup Ratio 的影響

【對應 RQ3，預期：部分符合，效應量遠小於預期】比較同一 workload 在 ASLR on / off 下的 dedup ratio 差異（見表 5-1），結果並未呈現單純「ASLR 開啟必然系統性降低 dedup ratio」的單一方向效應：

- Media Streaming 10 pods：ASLR on 反而略高（16.75% vs 16.39%，+0.36 pp）
- Media Streaming 30 pods：ASLR off 略高（17.85% vs 17.59%，+0.26 pp）
- Hotel Reservation 12 pods：ASLR off 明顯較高（20.78% vs 18.93%，+1.85 pp）
- Python API Farm 10 pods：ASLR off 略高（16.54% vs 16.16%，+0.38 pp）

四組對照中，三組呈現 ASLR off 時 dedup ratio 較高的方向（差異 0.26–1.85 個百分點），與理論假設（ASLR 引入的指標隨機性會降低內容重複率）方向一致，但唯一的例外（Media Streaming 10 pods）差異方向相反且量級極小（0.36 pp）。整體而言，**ASLR 對 dedup ratio 的影響方向大致符合假設，但效應量遠小於 workload 類型本身的差異**（Hotel Reservation 與 Media Streaming 之間的差距達 4 個百分點以上）。

這個效應量偏小的現象，可以用 `stress-ng --vm` 壓力測試手法本身的特性來解釋：在極端記憶體壓力下，被擠入 zswap 的頁面組成可能以 **pure data page**（資料 / buffer 頁面）為主，而真正包含指標、受 ASLR 影響的 metadata 頁面（如 stack frame）在所有進入 zswap 的頁面中占比可能微乎其微——這與第 5.4 節的頁面類型分布並不矛盾：PTR_HEAVY 類型雖然占多數，但

「指標密集」不等於「指標值因 ASLR 而隨機」，PTR_HEAVY 的判斷標準只看頁面是否大量出現 canonical pointer 格式的位元組模式，並未檢查這些指標的具體數值是否在 ASLR 開關之間發生變化。換言之，本研究目前的分類方法尚無法直接量化「同一份 PTR_HEAVY 內容在 ASLR 開／關下，有多少筆是因為指標數值不同而被判定為不同 hash」，這是 ASLR 效應量看起來偏小的可能原因之一，也是後續可以強化量測工具的方向。

換言之，決定 swap 內容重複率的主要因素是 workload 的架構特徵（共享多少 base image / runtime / 平台層 metadata），ASLR 的隨機化在高記憶體壓力場景下僅造成邊際的二階效應，RQ3 得到「方向上成立、但效應量遠小於預期」的結論。

一個值得注意的交叉現象是：**Media Streaming 30 pods 在 ASLR on 條件下，全零頁面熱點雜湊的觸發次數（78,024）遠高於 ASLR off（46,712）**，幾乎是 1.67 倍，且是全部 8 組量測中該雜湊出現次數的最高值。由於此全零頁面被推測與應用層內容無關（見 6.2 節），這個差異更可能反映 ASLR 開啟時系統整體記憶體配置模式變得更分散、進而提高了該系統性全零頁面被觸發 swap 的頻率，而非 ASLR 直接改變了該頁面的內容本身。此假設同樣需要 swap-in 路徑的追蹤資料才能進一步驗證。

7. 方法設計與實驗結果：VM 端

本章對應 RQ1–RQ3 在 VM 場景下的量測，用以檢驗容器端（第 4–6 章）的發現是否能推廣至虛擬機環境，並回答「容器與 VM 是否有系統性差異」這個問題。由組員負責設計與撰寫，以下先列出目前已完成的環境建置基礎，並標示待補充的章節骨架。

7.1 已完成的環境基礎

VM 端測試環境已完成以下建置，可作為後續正式量測的起點：

- 使用 KVM 搭配 Ubuntu Cloud Image 建立多 VM 環境，已驗證可建立 2 個 VM（各配置 2GB 記憶體），並透過 swapfile 啟用 swap
- 已確認目標 VM kernel 支援 CONFIG_ZSWAP=y，並可正常啟用 zswap
- 已透過 stress-ng --vm 在 VM 內產生記憶體壓力，並確認 stored_pages 計數器有對應增長
- 已完成初步測試：兩個環境設定相同的 VM 執行相同的 mmap 配置測試程式，將兩者的 core file dump 下來，以 4KB 為單位切分並計算 hash，量測頁面內容的相同程度，作為與 eBPF 動態量測結果的交叉驗證
- 已使用容器端開發的 eBPF 量測工具，在 host 端對兩個 VM 在 ASLR on/on 與 off/off 兩種對照設定下進行初步量測

7.2 方法設計

為驗證容器環境中的 deduplication 行為是否同樣適用於虛擬機環境，本研究於 Host 建立兩個 Ubuntu KVM Virtual Machine（VM），每個 VM 配置 2 GB 記憶體，並於 Guest OS 與 Host 端中啟用 zswap，並同樣利用 eBPF 掛載於 zswap_store()，即時監測進入 zswap 的 page。

與容器實驗不同，VM 的 Guest OS 擁有獨立的 kernel 與虛擬位址空間，因此需重新驗證：

Guest ASLR 是否仍影響 page deduplication

VM 是否與 container 呈現相同 dedup 行為

Guest workload 是否仍會造成 Host 端 page sharing

因此所有 dedup 統計皆於 Host 端完成，避免僅觀察 Guest memory 而忽略 Host zswap 的實際行為。

7.3 實驗設計

7.3.1 實驗假設

H1

若兩個 VM 執行完全相同之 workload，且 Guest ASLR 關閉，則大量 page 將具有相同內容，Host 端 duplicate ratio 將明顯提高。

H2

當 Guest 啟用 ASLR 後，由於 mmap 配置位置改變，page 對齊將受到影響，因此 duplicate ratio 將下降。

H3

採用 random workload 後，大量固定內容 page 將消失，剩餘 duplicate page 主要來自 Guest OS 本身之 zero page 與 kernel page。

7.3.2 Workload 來源與合理性

容器實驗主要採用實際 Web Application（Media Streaming、Hotel Reservation、Python API）作為 workload。

然而於 VM 初步驗證階段，並且由於 VM 端與 Container 端記憶體層級不同，包括 guest OS、Unused guest memory 等等，因此本研究先採用自行開發之 python 分配記憶體空間作為 guest VM 之 workload 將不同應用程式所造成的影響降低至最小，更能了解若是在相同環境下 VM 端的記憶體 swap 行為以及記憶體內易產生 duplicate 的內容，未來再考慮做 Container 相同的 workload 進行量測

7.3.3 環境設定與參數選擇

Host：

Ubuntu Server

RAM：8 GB

啟用 zswap

eBPF 掛載於 zswap_store()

VM：

VM1：Ubuntu Cloud Image，RAM 2 GB

VM2：Ubuntu Cloud Image，RAM 2 GB

Guest workload：

VM1：1500 MB random page

VM2：1500 MB random page

Host Memory Pressure :

使用 mempressure.c

逐步增加 2 GB、3 GB、5 GB 等不同壓力

觀察何時開始進入 zswap

ASLR :

ON / ON

OFF / OFF

ON / OFF

三組配置。

7.3.4 統計顯著性確保

未保持實驗的統一性與準確性，在每次量測前進行初始化，初始化流程如下:

1. 停止前一次 workload

關閉 VM 中執行的 testmem.py 或 stress-ng

停止 Host 端 stress-ng。

結束 eBPF 監測程式。

2. 清除 Swap 狀態

重新建立 swap 空間，清除上一輪實驗殘留於 zswap 中的 page。

3. 清除 Page Cache

清除 page cache、dentry 及 inode cache，降低快取造成的干擾。

4. 確認記憶體狀態及 ASLR 設定

5. 重新啟動 eBPF 監測工具

6. 啟動 VM Workload

7. 施加 Host 記憶體壓力

8. 方法設計與實驗結果：自製壓力測試程式

本章對應 RQ4：排除 stress-ng 的不可控變因後，自訂的壓力寫入模式與多 workload 啟動時間差，是否會改變量測到的 dedup ratio？由組員負責設計與撰寫。

8.1 動機：為何需要自製壓力測試工具

容器端與 VM 端目前皆採用 stress-ng --vm-bytes 產生記憶體壓力，其優點是能快速、穩定地鎖定大量匿名記憶體，逼迫系統觸發 swap-out；但其寫入模式（預設可能是固定 pattern 或偽隨機資料）並非實驗者可以精細控制，這對於想要進一步驗證「資料寫入模式本身如何影響 dedup ratio」的問題並不理想——若 stress-ng 內部的寫入演算法本身就帶有某種重複結構，量測到的 dedup ratio 可能部分來自工具本身的 artifact，而非真實應用程式的記憶體行為。

開發自製壓力測試程式的目的，即是取得對寫入內容與時間軸的完全控制權，用以驗證或排除這個疑慮，並補強 RQ1–RQ3 的結論穩健性。

8.1.1 容器端補充量測：host 端亂數壓力源敏感度分析

為了檢查原始容器端量測是否受到 stress-ng 內部資料型態影響，本研究在容器端額外補做一組解耦實驗：Media Streaming pod 內只保留純應用負載，client 以 ApacheBench 對 server 維持 300 秒請求；記憶體壓力則改由 host 端自製 C 程式產生。該程式以 mmap() 配置指定大小匿名記憶體，逐頁從 /dev/urandom 寫入 4 KB 不可壓縮亂數，讓壓力源本身盡可能不提供可去重內容。

此補充量測仍沿用第 3 章的 eBPF zswap_store hook，因此量測口徑與前述容器端實驗一致；差異僅在於壓力源由 pod 內 stress-ng 改為 host 端亂數壓力程式。這組實驗的目的不是取代第 5 章結果，而是提供一個較保守的對照：若在壓力源幾乎不可去重的條件下仍可觀察到明顯重複率，則更能支持容器基礎設施與 workload 冷頁面本身具有去重潛力。

表 8-1a 容器端補充量測資源配置

負載名稱	Pod 規模	Mem Request / Pod	Mem Limit / Pod	叢集總 Request 預估	叢集總 Limit 預估
Media Streaming (C++)	10 pods	400 Mi	1 Gi	~3.9 Gi	~9.8 Gi
Media Streaming (C++)	30 pods	250 Mi	300 Mi	~7.3 Gi	~8.8 Gi

表 8-1b host 端壓力參數與容器端去重率敏感度

Workload 規模	host 端壓力參數	Total pages	Dedup ratio	Memory saved	壓力狀態
10 pods	./stress 2500M 300	63,064	13.19%	32.50 MB	中度壓力 / swap onset
30 pods	./stress 2900M 300	54,985	12.45%	26.74 MB	中度壓力 / swap onset
30 pods	./stress 4500M 300	197,885	5.57%	43.08 MB	高度壓力 / sustained

					swap
10 pods	./stress 4500M 300	364,088	2.59%	36.89 MB	極限壓力 / deep thrashing

結果顯示，在壓力剛足以觸發 swap 時，10 pods 與 30 pods 的 dedup ratio 分別為 13.19% 與 12.45%，仍高於 Initial Guide 設定的 10% 決策門檻，也明顯高於既有 Android 單機場景約 5% 的基準。當 host 端亂數壓力提升後，dedup ratio 下降至 5.57% 與 2.59%；此下降較合理的解讀是：大量不可去重的亂數頁面進入 zswap 後擴大 total pages 分母，稀釋了容器冷頁面的重複比例。

表 8-1c 補充量測中的重複頁面類型分布

案例	ZERO	SAME_FILLED	TEXT/ASCII	PTR_HEAVY	BINARY
10 pods / 2500M	61.4%	12.3%	18.3%	5.3%	2.7%
30 pods / 2900M	54.8%	15.6%	18.5%	8.5%	2.4%
30 pods / 4500M	52.7%	9.8%	22.0%	11.7%	3.8%
10 pods / 4500M	60.3%	11.7%	19.0%	6.2%	2.7%

類型分布顯示 ZERO 在四組補充量測中皆占過半，TEXT/ASCII 穩定落在約 18%–22%，SAME_FILLED 約 10%–16%。這與第 6 章觀察到的全零頁面 thrashing 現象互相呼應，也表示在純亂數 host 壓力下，重複頁面仍主要來自系統性配置模式與容器平台資料，而非壓力工具本身創造出的重複內容。整體而言，容器端補充量測支持較精確的結論：容器環境在早期冷頁面回收階段具有明確去重潛力，但整體 dedup ratio 會隨壓力來源與 swap 深度動態改變。

8.2 方法設計

自製記憶體壓力產生工具（mempressure.c）

由於 Linux 常用的記憶體壓力工具（如 stress-ng）主要目的是對系統施加記憶體負載，其記憶體初始化方式並非為 deduplication 研究而設計，因此可能產生大量固定內容（例如 zero page、same-filled page 或固定 pattern），進而影響 zswap dedup ratio 的量測結果。

因此，本研究自行設計並實作 mempressure.c 作為記憶體壓力產生工具，其設計目標如下：

可控制配置大小：支援使用者指定配置記憶體容量（例如 512 MB、1 GB、2 GB、5 GB）。

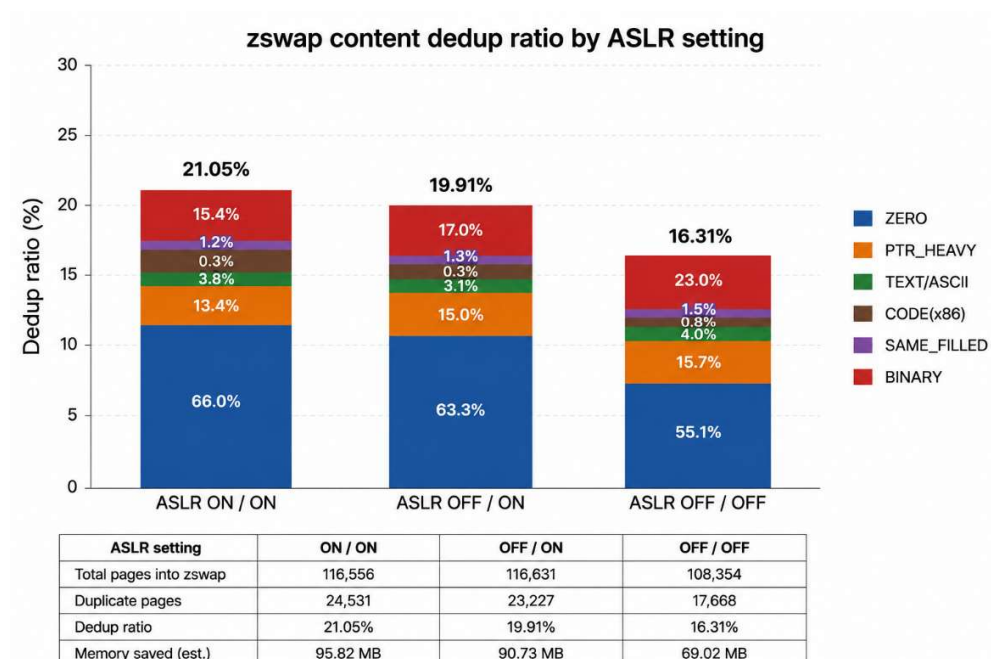
以 mmap() 配置匿名記憶體：建立與一般使用者程式相近的記憶體配置方式。

隨機初始化每個 page：以 /dev/urandom 對每個 4 KB page 寫入隨機內容，避免大量 zero page 或 same-filled page。

長時間維持記憶體配置：配置完成後持續保留記憶體，不進行額外 page 存取，以模擬實際應用程式長時間佔用記憶體之情境。

可控制執行時間：方便不同實驗條件下進行重複量測。

8.4 實驗結果



由整體結果可觀察到，三組實驗皆存在約 16% - 21% 的 duplicate ratio，且三者差異並不大。其中，ASLR ON/ON 組具有最高的 duplicate ratio，而 ASLR OFF/OFF 組則為最低，與原先假設「關閉 ASLR 將增加相同 page、進而提升 duplicate ratio」並不一致。表示在 VM 環境下，Guest ASLR 並非決定 Host-level zswap deduplication 的唯一因素。

進一步分析 Page Type Distribution 可發現，三組實驗中 ZERO page 均為主要的 duplicate page 類型，其比例分別為 66.0%、63.3% 與 55.1%。此外，PTR_HEAVY 與 BINARY page 亦佔有一定比例，而 SAME_FILLED page 則維持於 1% 左右。由於本研究已採用自行實作之 stress.c，以 /dev/urandom 對每個 4 KB page 進行隨機初始化，因此 SAME_FILLED page 的比例大幅降低，表示壓力工具本身已不再造成大量固定 pattern page，證明自製 workload 已有效降低 stress-ng 可能帶來的量測偏差。

另一方面，雖然三組實驗的 duplicate ratio 接近，但 duplicate page 的組成卻存在差異。ASLR ON/ON 與 OFF/ON 組中，ZERO page 均佔超過六成，推測目前 Host zswap 所觀察到的重複頁面，大部分來自 Guest OS 尚未使用的記憶體頁面，而非應用程式本身配置的 random page。相較之下，OFF/OFF 組中 PTR_HEAVY 與 BINARY page 的比例有所增加，顯示系統回收的 page 類型開始由未使用記憶體逐漸轉向程式執行期間所配置之資料頁面。

綜合以上結果，本研究推測目前 duplicate ratio 主要受到 Guest OS 記憶體管理、Linux memory reclaim policy 以及進入 zswap 的 page 組成所共同影響，而 Guest ASLR 對整體 duplicate ratio 的影響相對有限。由於 zswap 僅對實際被 swap out 的 page 進行 deduplication，而不同實驗中被回收之 page 類型及比例可能有所不同，因此即使 workload 與 Host memory pressure 已盡可能保持一致，仍可能因 Linux reclaim 行為的差異而造成 duplicate ratio 的變動。

因此，本研究認為 Host-level duplicate ratio 並不足以完全反映 ASLR 對應用程式 page similarity 的影響。若欲進一步分析 ASLR 對應用程式層級 page 的實際貢獻，未來可考慮排除 ZERO page 的影響，或將 duplicate page 依據 page type 分別統計，建立 Application-level duplicate Ratio，以降低 Guest OS 未使用記憶體對整體 duplicate 統計造成的干擾。

9. 研究限制與後續方向

9.1 量測工具的固有限制

9.1.1 eBPF 動態量測工具

以下限制是站在 eBPF 工具開發者的視角整理：本工具最初為容器端的量測需求而設計，但其 hook 點（zswap_store() 進入點）與輸出格式並不限定於特定虛擬化形式，VM 端沿用同一套工具進行量測時，以下限制同樣適用，且 host/guest 的程序情境問題可能使部分限制更為顯著。

- Comm 欄位不代表頁面擁有者：kprobe 在 zswap_store() 取得的程序情境是執行 swap-out 路徑的程序（常為 kswapd），而非頁面實際的擁有者（page owner）。本研究的 comm 統計（如「k3s-server 觸發 X 次」）僅能反映「誰觸發了這次 swap-out 呼叫」，不能直接推論「這個頁面屬於哪個程序的記憶體」。在 VM 場景下，這個限制可能進一步疊加 host/guest 視角混淆的問題：host 端的 kprobe 看到的程序情境，可能是 host kernel 自身的 kswapd，而非 guest VM 內部觸發記憶體壓力的程序，需要額外留意。
- 內容分類為啟發式、非精確分類：頁面類型分類（ZERO / PTR_HEAVY / TEXT_ASCII 等）僅依賴 4 個取樣位置共 128 bytes 的內容特徵，是一種快速篩選機制，不能保證每一筆分類完全正確；PTR_HEAVY 的判斷僅檢查位元組模式是否符合 canonical pointer 格式，並未檢查指標數值本身是否因 ASLR 而改變，這也是第 6.3 節 ASLR 效應量偏小的可能原因之一。
- Hash 碰撞抽樣驗證而非逐筆驗證：第 4.6 節說明的 byte-for-byte 抽樣驗證，受限於 BPF 執行環境的效能考量，並非對每一次 hash 命中進行驗證。FNV-1a 64-bit 在本研究的樣本規模下（單組量測約 100–300 萬頁面）碰撞機率極低，但理論上無法完全排除。
- 頁面類型統計範圍為重複頁面而非全部頁面：受 kernel 6.8 的 BPF verifier instruction count 限制（見 3.5 節），頁面分類邏輯移至 userspace，僅對觸發 ringbuf 事件的重複頁面進行分類，未涵蓋全部 swap-out 頁面（包括僅出現一次的 unique 頁面）。
- 取樣位置數量有限：目前僅在頁面的 4 個固定位置（offset 0、1024、2048、3072）各取樣 32 bytes 進行內容分類，覆蓋率僅占整個 4096 bytes 頁面的 3.125%，可能遺漏取樣點之間的内容變化，影響分類準確度。這也是第 9.3 節提出的後續改進方向之一。

9.1.2 自製壓力測試工具

雖然本研究所開發之 mempressure.c 已能提供可控制的記憶體配置大小、隨機資料初始化以及持續性的記憶體壓力，降低 stress-ng 對 dedup 量測可能造成的影響，但仍有數個值得進一步改善之方向。

首先，目前工具主要採用 一次性寫入（write-once）的方式建立 workload，配置完成後便維持記憶體內容不再更新。然而，實際應用程式通常伴隨持續性的資料新增、修改與釋放，因此未來可加入週期性更新部分 page 或可設定更新比例，以模擬真實應用程式長時間執行時的記憶體存取行為，進一步觀察動態 workload 對 zswap deduplication 的影響。

其次，目前 page 內容主要以 完全隨機（Random）或固定模式建立，未來可支援更多可配置的內容生成模式，例如：

Zero page

Same-filled page

Sequential pattern

Random page

Partial duplicate page（部分重複）

Configurable duplicate ratio（可指定重複比例）

透過控制 page 內容相似度，可更細緻地分析 deduplication 對不同資料特性的敏感程度。

此外，目前工具僅能控制配置容量與執行時間，未來亦可增加更多實驗參數，例如：

Page 更新頻率（Update Interval）

Memory Access Pattern（Sequential / Random Access）

Page Hotness（Hot / Cold Page）

執行緒數（Multi-thread）

NUMA Node 指定

Huge Page 支援

使工具更貼近不同應用程式之記憶體行為。

9.2 記憶體區段對應與動態追蹤：跨虛擬化形式的未完成項目

RQ2（重複內容歸因）目前僅部分達成——本研究能夠從頁面內容推斷資料類型（heap、應用層字串等），但無法將其對應到具體 process 的具體記憶體區段（如 /proc/[pid]/maps 中的哪一段 VMA）。如第 6.2 節所述，這是因為 zswap_store() 的程序情境不等於頁面擁有者。

若欲完成此目標，理論上的解法是在 swap-in 路徑（do_swap_page()）額外掛載第二個 kprobe：此時的程序情境即為頁面的實際擁有者，且仍能取得完整的 VMA 資訊。可用 swap entry（pte_to_swap_entry() 取得的 swap offset）作為 key，將 swap-out 與 swap-in 兩條路徑的事件在 userspace 端進行配對，進而還原「這個重複內容對應到哪個 process 的哪一段記憶體」。本研究尚未實作此路徑，留待後續擴充。

若進一步搭配 GDB 動態追蹤（attach 到固定、可重現的 virtual address 設定 watchpoint），鑑於本研究已發現一個跨 8 組量測一致出現的全零頁面熱點（見第 6 章），該熱點將是理想的首選追蹤目標。

9.3 VM 端 workload 完善

本研究於 VM 端已完成初步實驗平台建置，並以自製記憶體壓力程式產生可控制之 synthetic workload，用以排除 stress-ng 可能造成之固定 pattern 或 same-filled page 干擾。然而，目前 VM 端 workload 仍以人工產生的記憶體內容為主，尚未完全反映真實服務在虛擬機環境中的記憶體使用行為，因此未來仍需進一步補強 workload 設計。

首先，後續研究可將容器端已使用之實際應用 workload 移植至 VM 環境中，例如 Media

Streaming、Hotel Reservation 或 Python API Farm 等服務，使 VM 與 container 能在相同應用情境下進行比較。透過相同 workload 的跨環境部署，可更公平地分析 container 與 VM 在 page similarity、dedup ratio 及 page type distribution 上的差異。

此外，後續研究亦可設計多種 workload 類型，以區分不同應用特性對 dedup 的影響。例如 CPU-bound、memory-intensive、I/O-intensive、cache-heavy 與 database-backed workload 可能產生不同的 page composition。透過比較不同 workload 類型，可進一步釐清 VM 端 dedup ratio 主要來自應用程式資料、runtime library、Guest OS，或未使用記憶體頁面。

另一方面，為降低實驗間非決定性因素的影響，VM workload 的啟動流程、預熱時間、請求速率、資料集大小與量測時間應進一步標準化。尤其在 Host-level zswap 量測中，實際進入 zswap 的 page 會受到 Linux memory reclaim policy 影響，因此若 workload 狀態不一致，可能導致不同實驗間 page composition 差異過大。未來可透過固定 warm-up phase、steady-state 量測區間與重複實驗次數，以提高結果的穩定性與可比性。

最後，目前 VM 端實驗主要用於建立基準與觀察 ASLR 對 dedup 的初步影響。後續若能導入完整應用 workload，並同時區分 Overall Dedup Ratio 與 Application-level Dedup Ratio，將能更準確分析 ASLR、Guest OS 與 workload 本身對 zswap deduplication 的實際貢獻

9.4 後續研究方向

- 實作 `do_swap_page()` 的 swap-in 對應 hook，確認第 6.2 節提出的全零頁面成因假設，並完成記憶體區段對應與動態追蹤
- 透過控制變因實驗（例如固定微服務數量、僅改變 runtime 語言）拆解第 5.2 節觀察到的 workload 間差異，釐清 runtime 特性、應用層配置模式、微服務拓樸三者對 dedup ratio 的個別貢獻
- 完成 VM 端（第 7 章）對 RQ1–RQ3 的正式量測，與容器端結果交叉比對，驗證 RQ3 提出的「VM 受 ASLR 影響較小」假設是否成立，並檢驗 RQ1、RQ2 的結論是否能推廣至虛擬機環境
- 完成自製壓力測試工具（第 8 章）的開發與量測，以已知合成重複率的資料驗證 eBPF 量測工具本身的準確性，並更嚴謹地驗證 ASLR 效應與時間交錯效應
- 嘗試在容器端與 VM 端兩種環境下，將壓力測試的啟動時間錯開（而非同時啟動所有副本），觀察時間軸上的交錯如何影響 dedup ratio 與系統 swap 行為；此實驗結果可與第 5.3 節提出的 LRU 冰山模型對照討論——時間錯開是否等同於延長了觀察窗口、因而更容易觸及 LRU 深層的共用頁面
- 將 eBPF 量測工具的內容取樣位置從目前的 4 個固定位置（覆蓋率 3.125%）增加到更多位置，以提高頁面內容分類的準確度，並降低因取樣點之間內容變化被遺漏而導致的分類誤差；亦可評估改為更密集、等距分布的取樣策略，取代目前的 4 點取樣設計
- 若上述量測持續支持 content dedup 的工程價值，可進入 Initial Guide 階段二，於 `zswap_store()` 路徑加入實際的 dedup index 與 reference counting 機制，並評估 shrinker eviction 路徑的修改方案

延伸第 8.1.1 節的容器端補充量測，後續可在容器端加入 `memcg`、`cgroup path` 或 `swap-in` 配對資訊，將 `host` 壓力源頁面與容器 `workload` 頁面分開統計，進一步確認 12%–13% 早期 `swap` 去重率中各來源的實際貢獻

10. 結論

本研究圍繞 4 個研究問題，由兩位成員分工，透過共用的 `eBPF` 動態量測工具完成容器端與 VM 端的對應量測，並以自製壓力測試工具驗證結論的穩健性，系統性地驗證 `zswap` 內容重複率在多副本虛擬化環境下的具體數值與成因。本報告完整呈現容器端對 RQ1–RQ3 的方法、實驗設計與結果；第 8 章已先補入容器端使用自製 `host` 端亂數壓力程式的敏感度量測，其餘 VM 端與組員負責章節仍保留站位符，待正式結果完成後併入。

容器端的主要結論如下：

- 全部 8 組量測的 `dedup ratio` 介於 16.16%–20.78% 之間，穩定超過 Initial Guide 設定的 10% 決策門檻，且達既有文獻 Android 單機基準（5%）的三倍以上，確認容器化多副本環境存在顯著高於單機場景的 `content dedup` 機會（RQ1：成立）
- 重複頁面的內容組成以 `PTR_HEAVY`（`heap/stack` 結構，41.5%–61.5%）為主，`TEXT/ASCII`（容器平台 `metadata` 與監控資訊，14.7%–20.5%）次之，顯示重複來源主要是執行環境本身的記憶體配置模式；但 `workload` 間的差異方向與「Runtime & Memory Layout Characteristics」的原始預期不完全一致（第 5.2 節），三者（`runtime` 特性、應用層配置、微服務拓樸）的個別貢獻尚待控制變因實驗拆解（RQ2：部分成立，需後續驗證）
- 發現一個跨全部 8 組量測一致出現的系統性全零頁面 `thrashing` 熱點，其重複觸發次數占對應量測 `thrashing` 事件總量的 13.9%–34.9%，是本研究最值得後續深入追蹤的具體現象
- `ASLR` 對 `dedup ratio` 的影響方向大致符合「開啟 `ASLR` 降低重複率」的假設，但效應量（0.26–1.85 個百分點）遠小於 `workload` 類型本身造成的差異（4 個百分點以上），可能原因是高壓 `swap` 場景下進入 `zswap` 的頁面以 `pure data page` 為主（RQ3：方向成立，效應量遠小於預期）

容器端補充量測顯示，使用 `host` 端不可壓縮亂數壓力源排除 `stress-ng` 自我去重干擾後，`Media Streaming workload` 在早期 `swap` 階段仍有 12.45%–13.19% 的 `dedup ratio`；但在深度壓力下會降至 5.57% 或 2.59%，表示容器端 `dedup` 效益最明顯的區間是早期冷頁面回收階段，而非任意極端壓力下的固定比例

容器端量測工具本身在擴充過程中，記錄了遭遇的 `BPF verifier instruction count` 限制問題，以及對應的架構調整（頁面分類邏輯移至 `userspace`），這部分的工程經驗已整理於工程筆記（`engineering_notes.md`）中，可作為後續以 `eBPF` 進行 `kernel` 動態追蹤研究的參考。

綜合而言，容器端的量測結果支持 `zswap content dedup` 具有足夠的工程價值，值得進入 Initial Guide 階段二的 `kernel` 修改階段；同時，本研究發現的系統性全零頁面 `thrashing` 現象，本身亦是一個獨立、具體、可重現的研究發現。容器端補充量測進一步說明，若排除 `stress-ng` 自身資料型態干擾，早期 `swap` 階段仍可觀察到 12%–13% 的去重率，但極端外部亂數壓力會稀釋整體 `dedup ratio`。VM 端與組員負責的自製壓力測試結果待補齊後，將是判斷上述結論是否能推廣至其他虛擬化形式、是否能排除既有測試工具 `artifact` 的關鍵佐證。

VM 端主要結論如下：

本章針對 RQ1–RQ4 於 VM 場景進行初步量測。整體而言，實驗結果顯示，在 Host-level zswap 觀察下，VM 環境確實存在可觀察到的 page-level content deduplication，但其來源並不單純來自應用程式 workload，而是受到 Guest OS、zero page、memory reclaim policy 與 ASLR 設定共同影響。

對應 RQ1，本實驗在 VM1 與 VM2 皆配置 2 GB RAM，並執行相同 random workload 的條件下，量測到約 **16%–21%** 的 zswap dedup ratio。此數值高於 Android 單機基準約 5% 的參考值，表示在多 VM 環境中，即使 workload 使用 random page，Host 端仍可觀察到一定程度的重複頁面。然而，從 page type distribution 可發現，重複頁面中有相當比例來自 **ZERO page**，因此該 dedup ratio 不應直接解讀為應用程式本身產生高度重複，而應視為 VM 整體記憶體狀態下的 host-level dedup 結果。

對應 RQ2，實驗結果顯示重複頁面的主要來源並非 random workload 本身，而是 Guest OS 與虛擬化環境中未被 workload 覆蓋的記憶體頁面。三組 ASLR 設定下，ZERO page 皆為主要 duplicate page 來源，比例約為 **55%–66%**。此外，PTR_HEAVY 與 BINARY page 也佔有一定比例，表示除了未使用記憶體外，部分重複頁面可能來自程式 runtime、library、QEMU 或 Guest kernel 相關資料結構。這說明 VM 場景中的 dedup 來源比 container 更複雜，不能僅從應用程式 workload 判斷。

對應 RQ3，ASLR 對 dedup ratio 的影響在本次 Host-level zswap 量測中並非呈現單一方向的明顯趨勢。三組結果分別為：ASLR ON/ON 約 **21.05%**、OFF/ON 約 **19.91%**、OFF/OFF 約 **16.31%**。雖然 OFF/OFF 並未如 H1 預期產生最高 dedup ratio，但整體差異仍顯示 ASLR 設定會改變進入 zswap 的 page composition。由於 ASLR 主要改變 mmap、heap、stack 等記憶體配置位置，而 zswap dedup 則取決於實際被 reclaim 的 page content，因此 ASLR 的影響可能被 memory reclaim policy、zero page 比例與 Guest OS 記憶體狀態稀釋。此結果也部分修正 H1 與 H2：ASLR 關閉不必然提高 host-level dedup ratio，實際結果仍取決於被換出頁面的內容組成。

對應 RQ4，本研究改以自製 mempressure.c 取代 stress-ng，以避免 stress-ng 可能產生固定 pattern 或大量 same-filled page 造成 dedup ratio 偏高。實驗中 random workload 使 SAME_FILLED page 比例維持在低比例，顯示自製壓力工具確實降低了固定內容 page 對量測的干擾。然而，ZERO page 仍然佔據主要 duplicate 來源，表示即使排除 stress-ng 的影響，Host-level zswap 仍會受到 Guest unused memory 與系統層級 page 的影響。

綜合而言，本章實驗結果支持 H3：採用 random workload 後，大量固定 pattern page 確實被消除，剩餘 duplicate page 主要來自 Guest OS 的 zero page 與系統層級頁面。相較之下，H1 與 H2 在 Host-level zswap 量測下未被完全支持，因為 ASLR 對 dedup ratio 的影響並非單獨決定因素。後續若要更精確評估 ASLR 本身的影響，應進一步排除 ZERO page，並將 dedup ratio 拆分為 Overall Dedup Ratio 與 Application-level Dedup Ratio，以觀察 PTR_HEAVY、BINARY、TEXT/ASCII 等非零頁面的變化。